Taller de Proyecto II

Investigación de Modelos: Cinemática Inversa y Detección de Autocolisiones

Grupo J1
Tarifa, Armando Ezequiel[1] (02893/4)
Guitera Woida, Emma[1] (03352/2)
Terruzzi, Ana Justina[1] (02557/9)

02/10/2026

[1]{ezequiel.tarifa.2002,emma.guitera.woida,justina.terruzzi}@gmail.com

---

Índice

1. Introducción 2
2. Modelos implementados heredados 2
   2.1. Cinemática Directa (FK) - Implementada en 2025 2
   2.2. Cinemática Inversa (IK) - No implementada en 2025 2
   2.3. Detección de Autocolisiones - Implementada en 2025 2
3. Cinemática Inversa Iterativa - Hito 1.2 3
   3.1. Definición del problema 3
   3.2. Algoritmos evaluados 3
      3.2.1. Jacobiano transpuesto 3
      3.2.2. Damped Least Squares (DLS) 3
      3.2.3. CCD (Cyclic Coordinate Descent) 3
   3.3. FABRIK (Forward And Backward Reaching Inverse Kinematics) 3
   3.4. Tabla comparativa 4
   3.5. Decisión: FABRIK 4
   3.6. Formalización matemática del algoritmo 4
      3.6.1. Paso 0 — Verificación de alcanzabilidad 4
      3.6.2. Paso 1 — Fase Forward (llevar la punta al objetivo) 4
      3.6.3. Paso 2 — Fase Backward (anclar la base) 5
      3.6.4. Restricciones dentro de cada paso 5
      3.6.5. Parámetros elegidos 5
   3.7. Recuperación de ángulos articulares 5
   3.8. Aplicación de FABRIK en nuestro brazo 6
4. Modelo de Esferas para Detección de Autocolisiones — Hito 1.3 7
   4.1. Contexto: el modelo del año anterior 7
   4.2. Adopción del mismo modelo 7
   4.3. Fundamentación matemática 7
   4.4. Distribución de esferas por eslabón 7
   4.5. Matriz booleana de exclusión 5×5 8
      4.5.1. Reglas de exclusión 8
      4.5.2. Exclusión de articulación compartida 8
   4.6. Colisión contra el plano base 8
5. Conclusión 9
Referencias 10

1. Introducción

En este documento de investigación se recopila la fundamentación teórica, comparación de algoritmos y decisiones de diseño correspondientes a los hitos **H1.2** (Cinemática inversa iterativa) y **H1.3** (Modelo de esferas para detección de autocolisiones) del proyecto J1.

A lo largo del informe se referencia lo que el grupo J5 del año 2025 implementó en su proyecto [1], para establecer qué se hereda directamente, qué se modifica y qué es nuevo en nuestro proyecto.

2. Modelos implementados heredados

Antes de avanzar con modelos nuevos, se mencionan los modelos implementados por el grupo anterior [1].

2.1. Cinemática Directa (FK) - Implementada en 2025

El grupo J5 adoptó la convención **Denavit-Hartenberg (DH)** como marco teórico para modelar la cinemática directa. La implementación en C resultante construye cada matriz T_{i-1}^{i} combinando una rotación de corrección de montaje (R_{fix}), la rotación del servo y la traslación del eslabón, lo que es equivalente al producto DH estándar.

La cadena cinemática del brazo de 5 grados de libertad se modela como:

$$T_{\text{total}} = T_0^1 \cdot T_1^2 \cdot T_2^3 \cdot T_3^4 \cdot T_4^5$$

La secuencia de ejes de rotación es **Z-Y-Y-X-Y** para los servos 0 a 4. Cada R_{fix} alinea el eje físico del servo con el eje Z matemático para que todas las rotaciones se expresen uniformemente.

Esta implementación fue codificada en C (`cinematica.c`, `algebra_lineal.c`, `numpy.c`) para el ESP32 y replicada en Python (`cinematica.py`, `algebra_lineal.py`) para simulación.

Se hereda para el proyecto 2026 sin modificaciones en la lógica de cinemática directa.

2.2. Cinemática Inversa (IK) - No implementada en 2025

El proyecto del 2025 **no implementó cinemática inversa**. El brazo se controlaba exclusivamente mediante un potenciómetro analógico y botones pulsadores: el operador seleccionaba un servo y lo rotaba manualmente. El sistema verificaba colisiones con cada pose resultante, pero no existía la posibilidad de enviar un destino (X, Y, Z) y que el sistema calculara automáticamente los ángulos necesarios.

La cinemática inversa se implementa en nuestro proyecto 2026.

2.3. Detección de Autocolisiones - Implementada en 2025

El grupo J5 implementó un sistema de detección de autocolisiones basado en **esferas delimitadoras** [1]. El modelo define 9 esferas distribuidas en los 5 eslabones con radio uniforme $r = 1.0$ cm, verificando pares de esferas filtrados por una matriz booleana 5x5 para evitar falsos positivos entre eslabones adyacentes.

Se hereda para el proyecto 2026.

3. Cinemática Inversa Iterativa - Hito 1.2

3.1. Definición del problema

Dado un punto objetivo $\boldsymbol{p}^* \in \mathbb{R}^3$ en el espacio de trabajo del brazo, el problema de la cinemática inversa consiste en encontrar el vector de ángulos articulares:

$$\boldsymbol{\theta}^* = (\theta_0^*, \theta_1^*, \theta_2^*, \theta_3^*, \theta_4^*)$$

tal que la evaluación de la cinemática directa resulte en dicho punto:

$$\mathrm{FK}(\boldsymbol{\theta}^*) = \boldsymbol{p}^*$$

Para un brazo de 5 grados de libertad con geometría no esférica, la resolución analítica cerrada es altamente compleja y propensa a errores de implementación debido a la cantidad de casos y condiciones. Por esta razón, se opta por un enfoque numérico iterativo capaz de ejecutarse eficientemente en el ESP32.

3.2. Algoritmos evaluados

Se estudiaron y compararon cuatro métodos de cinemática inversa iterativa:

3.2.1. Jacobiano transpuesto

Calcula el Jacobiano $J(\boldsymbol{\theta}) \in \mathbb{R}^{3\times n}$ y actualiza los ángulos con:

$$\Delta\boldsymbol{\theta} = \alpha \cdot J^T \cdot (\boldsymbol{p}^* - \mathrm{FK}(\boldsymbol{\theta}))$$

- **Ventajas:** No requiere inversión de matrices.
- **Desventajas:** Convergencia lenta (1311 iteraciones en promedio en la Tabla 1 del paper [2]), movimientos que pueden oscilar, requiere elegir α adecuadamente.

3.2.2. Damped Least Squares (DLS)

Resuelve el sistema regularizado:

$$\Delta\boldsymbol{\theta} = \left(J^T J + \lambda^2 I\right)^{-1} J^T \cdot (\boldsymbol{p}^* - \mathrm{FK}(\boldsymbol{\theta}))$$

- **Ventajas:** Robusto frente a singularidades gracias al parámetro de amortiguación λ.
- **Desventajas:** Requiere invertir una matriz $n \times n$ en cada iteración, costoso para el ESP32, y hay que elegir el parámetro λ (el paper usa un valor fijo, 1.1).

3.2.3. CCD (Cyclic Coordinate Descent)

Ajusta un ángulo por vez, recorriendo las articulaciones cíclicamente. Cada articulación i se rota para minimizar la distancia entre el efector y el objetivo en su plano de acción.

- **Ventajas:** Sin matrices, simple de implementar.
- **Desventajas:** Convergencia errática, no garantiza el camino más corto.

3.3. FABRIK (Forward And Backward Reaching Inverse Kinematics)

FABRIK es un algoritmo que resuelve la cinemática inversa de forma iterativa, ajustando sucesivamente las posiciones de las articulaciones para acercar el efector al objetivo. Opera con posiciones cartesianas de las articulaciones. Cada articulación se reubica sobre la recta que pasa por su posición anterior y por la nueva posición de la articulación vecina, a la distancia que fija el largo del eslabón [2], [3], [4].

- **Ventajas:** Sin matrices ni derivadas, convergencia en 5-20 iteraciones, movimientos naturales. Las restricciones articulares se pueden incorporar dentro del algoritmo, en cada paso de la iteración [2].
- **Desventajas:** Como el resultado son posiciones, hace falta un paso final para obtener el ángulo de cada servo. Además, cada tipo de articulación (bisagra, rótula) necesita su propia forma de restringirse.

3.4. Tabla comparativa

Criterio	Jac. Transpuesto	DLS	FABRIK	CCD
Invierte una matriz	No	Sí ($n \times n$)	No	No
Singularidades	Robusto (sin inversa)	Robusto (λ)	Inmune	Robusto
Tiempo por iteración (Tabla 1 del paper)	9.90 ms	10.49 ms	0.86 ms	4.69 ms
Convergencia típica	1311 iter.	999 iter.	5–20 iter.	26 iter.
Naturalidad del movimiento	Media	Media	Alta	Baja
Implementación en C para ESP32	Media	Alta (inversión)	Baja-Media	Baja
Adecuado para ESP32	Aceptable	Aceptable	**Óptimo**	Aceptable

3.5. Decisión: FABRIK

Se selecciona **FABRIK** como algoritmo de cinemática inversa para el proyecto 2026. La decisión se tomó por los siguientes motivos:

- **Sin inversión matricial:** Opera con sumas, restas y normalizaciones de vectores 3D. No necesita calcular jacobianos ni inversiones de matrices. Por lo tanto, su implementación en C es más simple y rápida que las otras alternativas.
- **Inmunidad a singularidades:** Al no depender de derivadas, no existe configuración del brazo que genere una matriz singular.
- **Convergencia rápida:** Aristidou & Lasenby reportan que, en sus pruebas, FABRIK resultó unas 10 veces más rápido que CCD y unas mil veces más rápido que los métodos basados en el Jacobiano [2].
- **Implementación real disponible:** Existe una librería de código abierto (NocKinematics) que implementa FABRIK en 3D para Arduino y ESP32, aunque sin límites articulares [5].
- **Movimiento natural:** Los trazados son suaves y predecibles.

3.6. Formalización matemática del algoritmo

Sean $\boldsymbol{p}_0, \boldsymbol{p}_1, ..., \boldsymbol{p}_n$ las posiciones globales de las $n+1$ articulaciones, con $\boldsymbol{p}_0$ siendo la base fija del brazo y $\boldsymbol{p}_n$ el extremo efector. Sean $l_i = \|\boldsymbol{p}_i - \boldsymbol{p}_{i-1}\|$ las longitudes mecánicas constantes de cada eslabón.

3.6.1. Paso 0 — Verificación de alcanzabilidad

Antes de iterar, se comprueba si el objetivo está dentro del rango máximo del brazo:

$$\|\boldsymbol{p}^* - \boldsymbol{p}_0\| \leq \sum_{i=1}^{n} l_i$$

Si no se cumple, el objetivo es inalcanzable y no se ejecuta ninguna iteración. En nuestro brazo, el error HTTP al Gemelo Digital se envía recién cuando ninguna posición de la muñeca y la pinza llega (Sección 3.8).

3.6.2. Paso 1 — Fase Forward (llevar la punta al objetivo)

Se coloca el efector directamente en el objetivo y se propagan las restricciones de longitud hacia la base:

$$\boldsymbol{p}_n \leftarrow \boldsymbol{p}^*$$
$$\boldsymbol{p}_{i-1} \leftarrow \boldsymbol{p}_i + \frac{\boldsymbol{p}_{i-1} - \boldsymbol{p}_i}{\|\boldsymbol{p}_{i-1} - \boldsymbol{p}_i\|} \cdot l_i \quad \text{para} \quad i = n, n-1, ..., 1$$

3.6.3. Paso 2 — Fase Backward (anclar la base)

Como la base quedó desplazada, se la devuelve a su posición fija $\boldsymbol{p}_{\text{base}}$ y se propagan las restricciones hacia adelante:

$$\boldsymbol{p}_0 \leftarrow \boldsymbol{p}_{\text{base}}$$
$$\boldsymbol{p}_i \leftarrow \boldsymbol{p}_{i-1} + \frac{\boldsymbol{p}_i - \boldsymbol{p}_{i-1}}{\|\boldsymbol{p}_i - \boldsymbol{p}_{i-1}\|} \cdot l_i \quad \text{para} \quad i = 1, 2, ..., n$$

Los pasos 1 y 2 se repiten alternadamente hasta que se cumple alguno de los tres criterios de corte que plantea el paper [2]:

$$\|\boldsymbol{p}_n - \boldsymbol{p}^*\| \leq \varepsilon \quad \text{o} \quad \left\|\boldsymbol{p}_n^{(k)} - \boldsymbol{p}_n^{(k-1)}\right\| < \eta \quad \text{o} \quad k > k_{\max}$$

El primero indica que el efector llegó al objetivo. El segundo compara la posición del efector en dos iteraciones seguidas. Si casi no se movió, seguir iterando no mejora el resultado. Por esta razón, es el criterio que corta la iteración cuando el objetivo no se puede alcanzar. El tercero es un límite de seguridad.

3.6.4. Restricciones dentro de cada paso

Los pasos anteriores corresponden a la forma sin restricciones, en la que cada articulación se comporta como una rótula libre. Las restricciones articulares se aplican dentro de la iteración y no después de converger. Cada vez que el forward o el backward reubica una articulación, se verifica que la orientación resultante esté dentro del rango permitido. Si no lo está, se la lleva a ese rango antes de calcular la siguiente. Según el paper, los algoritmos de cinemática inversa suelen ser más efectivos si las restricciones se aplican en cada paso y no al final. Además, FABRIK con restricciones llega a la solución siempre que el objetivo esté al alcance y exista una configuración de las articulaciones que permita llegar [2].

Para articulaciones de un solo grado de libertad, como una bisagra, el paper propone una simplificación. La raíz y el objetivo, junto con la orientación del objetivo, definen un plano. Las articulaciones se proyectan sobre ese plano y FABRIK se aplica en 2D dentro de él (sección 4.3 y Fig. 7 del paper).

Con restricciones, la verificación del Paso 0 es necesaria pero no suficiente. Un objetivo que está dentro del rango máximo puede quedar fuera de alcance por los límites de los servos. En ese caso el efector deja de acercarse y la iteración termina por el segundo criterio o, si la cadena sigue deslizándose contra un tope, por el tercero.

3.6.5. Parámetros elegidos

Parámetro	Valor	Justificación
ε (tolerancia)	0.5 cm	Resolución mecánica del SG90 (1° ≈ 0.33 cm en el extremo)
η (sin movimiento)	10^{-4} cm	Detecta que el efector dejó de moverse entre dos iteraciones
$k_{\max}$ (iteraciones)	20	Se reportan convergencias en 5–15 iteraciones para cadenas similares

3.7. Recuperación de ángulos articulares

FABRIK determina posiciones cartesianas $\boldsymbol{p}_i$, pero los servos necesitan ángulos. El ángulo θ_i es el giro alrededor del eje $\boldsymbol{z}_i$ que lleva al eslabón desde su posición de cero ($\theta_i = 0$) hasta la posición encontrada. Por lo tanto, se mide entre dos direcciones del plano perpendicular a $\boldsymbol{z}_i$:

$$\theta_i = \text{atan2}(\boldsymbol{z}_i \cdot (\boldsymbol{r}_i \times \boldsymbol{d}_i), \quad \boldsymbol{r}_i \cdot \boldsymbol{d}_i)$$

donde $\boldsymbol{d}_i = \boldsymbol{p}_{i+1} - \boldsymbol{p}_i$ es el vector director del eslabón, $\boldsymbol{z}_i$ es el eje de rotación del servo i expresado en coordenadas globales y $\boldsymbol{r}_i$ es la dirección que tendría el eslabón con $\theta_i = 0$. Los vectores $\boldsymbol{d}_i$ y $\boldsymbol{r}_i$ se toman proyectados sobre el plano perpendicular a $\boldsymbol{z}_i$. Tanto $\boldsymbol{z}_i$ como $\boldsymbol{r}_i$ se obtienen de la cadena FK con los ángulos ya recuperados de los servos anteriores, así que el cálculo avanza en orden, desde la base hacia la pinza. El signo surge del propio atan2, que da un valor positivo cuando el giro de $\boldsymbol{r}_i$ a $\boldsymbol{d}_i$ es antihorario visto desde la punta de $\boldsymbol{z}_i$.

Medir en cambio el ángulo entre el eslabón y su propio eje, $\text{atan2}(\|\boldsymbol{d}_i \times \boldsymbol{z}_i\|, \boldsymbol{d}_i \cdot \boldsymbol{z}_i)$, no sirve, porque una rotación alrededor de $\boldsymbol{z}_i$ no cambia el ángulo que forma un vector con $\boldsymbol{z}_i$. Con la geometría actual del modelo de nuestro brazo, ese cálculo da siempre $(0°, 90°, 90°, 180°, 90°)$, sin importar la pose.

Por último, los ángulos recuperados solo reproducen las posiciones de FABRIK si esas posiciones respetan la estructura del brazo. En el modelo actual, en el hombro y el codo cada eslabón tiene que quedar perpendicular a su eje, y en la base y la muñeca, alineado con él. Un eslabón alineado con su eje no cambia al girar, así que en la base y la muñeca el ángulo no se puede obtener de su propio eslabón. En nuestro brazo, el de la base surge de la orientación del plano del brazo y los de la muñeca y la pinza los elige el programa antes de iterar (Sección 3.8).

Los rangos de los servos son los que permite el firmware (`main.ino`), porque todavía no se midieron sobre el brazo. Se expresan en grados del servo, que el firmware obtiene a partir del ancho de pulso con sus tablas de calibración, y se convierten a los ángulos del modelo con la misma regla que usa el firmware:

Servo	Articulación	Rango del servo (°)	Conversión	Rango del modelo (°)
0	Base	0 a 180	$\theta_0 = -s_0$	−180 a 0
1	Hombro	0 a 180	$\theta_1 = s_1$	0 a 180
2	Codo	0 a 180	$\theta_2 = -s_2$	−180 a 0
3	Muñeca	0 a 180	$\theta_3 = s_3 - 90°$	−90 a 90
4	Pinza	0 a 180	$\theta_4 = s_4$	0 a 180

Recortar los ángulos recién después de converger no alcanza. El recorte cambia la pose y el efector deja de estar sobre el objetivo. Además, FABRIK sin restricciones puede converger a una solución que el brazo no puede hacer (por ejemplo, con el codo doblado hacia el lado contrario), aunque exista otra que sí respete los límites. Por eso los límites se aplican en cada paso, junto con una regla para el lado del codo (Sección 3.8), y, como control final, la pose se declara inválida si el efector quedó a más de ε del objetivo o si algún ángulo quedó fuera de rango.

3.8. Aplicación de FABRIK en nuestro brazo

Hasta aquí se describió FABRIK en su forma general. En nuestro brazo las articulaciones no son rótulas libres. La base gira alrededor del eje vertical, el hombro y el codo son bisagras de ejes paralelos, la muñeca rota sobre el eje del antebrazo y el servo de la pinza inclina hacia adelante y hacia atrás. Por esta razón, se va a aplicar una adaptación de la técnica que propone el paper para bisagras [2]. La base gira el plano vertical del brazo para que $\boldsymbol{p}_5$, que según la muñeca puede quedar fuera de ese plano, caiga sobre el objetivo. Dentro del plano, FABRIK resuelve el hombro y el codo en 2D, con los límites aplicados en cada paso. Si hay dos orientaciones posibles para la base, se prueba primero la que apunta hacia el objetivo. Para elegir entre las dos posiciones posibles del codo, y evitar que termine doblado hacia abajo, se agrega una regla similar al *pole* que usan programas de animación como Blender y Maya [6], [7]. El usuario solo indica el punto al que tiene que llegar la pinza. La muñeca y la pinza empiezan en su posición neutra y, si con ellas el punto queda fuera de alcance o bloqueado por los límites, el programa prueba otras posiciones de a 10°, primero en la pinza y después en la muñeca, hasta encontrar una que llegue.

Esta aplicación se encuentra documentada en detalle en el informe **Implementación FABRIK** (`implementacion-fabrik.pdf`).

4. Modelo de Esferas para Detección de Autocolisiones — Hito 1.3

4.1. Contexto: el modelo del año anterior

El grupo J5 (2025) diseñó e implementó un sistema de detección de autocolisiones basado en **esferas delimitadoras** [1]. El modelo se integró al firmware del prototipo 2025 para detectar colisiones entre eslabones y contra el plano base, y bloquear los movimientos peligrosos.

4.2. Adopción del mismo modelo

Se evaluaron otras alternativas, como **cápsulas** o **AABB / OBB**. Sin embargo, se concluyó que las esferas delimitadoras son la mejor opción para nuestro prototipo por las siguientes razones:

- **Ya está implementado:** El grupo J5 lo programó en C y lo integró al firmware del ESP32 [1].
- **Costo computacional bajo:** La evaluación de colisión entre dos esferas se reduce a una sola operación de distancia euclidiana.
- **Compatibilidad directa:** El código C heredado ya implementa este modelo. Solo hay que pasarlo a memoria estática, porque hoy usa `malloc`, `realloc` y `free`, que las reglas de firmware de la cátedra prohíben.
- **Suficiente para el caso de uso:** Los eslabones del brazo SG90 son cilindros cortos; las esferas proporcionan un margen de seguridad adecuado sin precisar geometrías más complejas.

Las cápsulas (Swept Sphere Volumes) serían una mejora futura si se detectan falsos negativos con el hardware real, por el momento se mantendrá el modelo de esferas.

4.3. Fundamentación matemática

La intersección entre dos esferas S_A y S_B en $\mathbb{R}^3$ se evalúa con una única desigualdad euclidiana [8]:

$$d(\boldsymbol{c}_A, \boldsymbol{c}_B) = \sqrt{(c_A^x - c_B^x)^2 + (c_A^y - c_B^y)^2 + (c_A^z - c_B^z)^2} \leq r_A + r_B + \delta$$

Para el prototipo, se usa radio uniforme $r_{\text{esf}} = 1.0$ cm (heredado del año anterior) y tolerancia numérica $\delta = 10^{-9}$ cm. La condición de colisión se simplifica a:

$$d(\boldsymbol{c}_A, \boldsymbol{c}_B) \leq 2 \cdot r_{\text{esf}}$$

4.4. Distribución de esferas por eslabón

Las coordenadas se definen en el espacio local de cada eslabón como vectores homogéneos $\tilde{c}_{i,j}^{\text{loc}} = (x, y, z, 1)^T$. La transformación a coordenadas globales reutiliza la misma cadena de matrices $M_{\text{eslabon}}^{(i)}$ calculada por la FK:

$$c_{i,j}^{\text{glob}} = M_{\text{eslabon}}^{(i)} \cdot \tilde{c}_{i,j}^{\text{loc}}$$

Eslabón	Articulación	Centros locales (x, y, z)	Cant.
0	Base	$(0, 0, 0)$	1
1	Hombro	$(0, 0, 0)$; $(2, 0, 0)$	2
2	Codo	$(0, 0, 0)$; $(2, 0.5, 0)$; $(4, 0.5, 0)$	3
3	Muñeca	$(0, 0, 0)$	1
4	Pinza	$(0, 0, 0)$; $(0, -2, 0)$	2
	Total		**9 esferas**

Estos valores son los documentados en `configuracion.c` *del prototipo 2025* [1]. *Serán validados contra el hardware físico real una vez ensamblado, ajustando posiciones y radio si corresponde.*

4.5. Matriz booleana de exclusión 5×5

Sin optimización, verificar colisiones entre las 9 esferas requeriría $\binom{9}{2} = 36$ evaluaciones de distancia en cada ciclo. La **Matriz de Colisiones** booleana de `configuracion.c` (heredada de [1]) filtra los pares que no hace falta verificar:

$$M_{\text{colisiones}} = \begin{pmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \end{pmatrix}$$

$M[i, j] = 1 \rightarrow$ verifica colisión entre eslabón i y eslabón j.

4.5.1. Reglas de exclusión

- **Diagonal** ($i = j$): Un eslabón no colisiona consigo mismo.
- **Adyacentes** ($|i - j| = 1$): Los eslabones consecutivos comparten una articulación física. Sus esferas extremas se superponen por diseño mecánico. Por eso no se verifican, salvo el par hombro–codo $(1, 2)$, porque el codo llega a $-180°$ y puede plegar el antebrazo sobre el brazo.
- **Casi-adyacentes** ($|i - j| = 2$): En los pares $(0, 2)$ y $(1, 3)$, con las medidas actuales, cada choque también lo detecta el chequeo contra el piso u otro par verificado. En el par $(2, 4)$, la última esfera del codo y la primera de la pinza quedan siempre a 2 cm, así que se tocarían en cualquier pose.
- **Base y muñeca** ($i = 0$, $j = 3$): La esfera de la muñeca coincide con la última del codo. Por eso sus choques con la base son los mismos que los del par $(0, 2)$.

4.5.2. Exclusión de articulación compartida

En el único par adyacente que la matriz sí verifica, el $(1, 2)$, se excluye además el cruce entre la última esfera del eslabón i y la primera del eslabón j, porque con las medidas actuales sus centros quedan siempre a 1.3 cm y se superponen en cualquier pose:

$$\text{Excluir: } \left(c_{i,\ \text{última}},\quad c_{j,\ \text{primera}}\right)$$

4.6. Colisión contra el plano base

Adicionalmente, cada esfera (excepto las del eslabón 0, que es la base) se verifica contra el plano del suelo ($Z = 0$):

$$c_{i,j}^{\text{glob}}[z] \leq r_{\text{esf}} \quad \forall i \in \{1, 2, 3, 4\}, \quad \forall j$$

Si cualquier esfera penetra el plano, la pose se rechaza inmediatamente.

5. Conclusión

La investigación establece que:

1) El modelo de **cinemática directa** (FK) del año anterior es matemáticamente correcto y se adopta sin cambios en su lógica. Sus medidas se reemplazaron por las del brazo armado, y `configuracion.c` se corrige en el pasaje a C.
2) El modelo de **esferas delimitadoras** para colisiones fue implementado por el grupo J5 y es el más adecuado para las restricciones del ESP32. Se adopta, y sus posiciones y radio se van a validar con el brazo armado.
3) **FABRIK** es el algoritmo óptimo para IK en nuestro contexto: sin matrices, convergencia rápida, implementación simple en C. Es la contribución técnica nueva del grupo J1.

Referencias

[1] Grupo J5 — Taller de Proyecto II 2025, «Brazo Robótico Seguro — Bitácora y documentación técnica». [En línea]. Disponible en: https://github.com/tpII/2025-J5-Brazo-Robotico-Seguro/wiki

[2] A. Aristidou y J. Lasenby, «FABRIK: A fast, iterative solver for the Inverse Kinematics problem», *Graphical Models*, vol. 73, n.º 5, pp. 243-260, 2011, [En línea]. Disponible en: https://www.andreasaristidou.com/publications/papers/FABRIK.pdf

[3] A. Aristidou, «FABRIK: A fast, iterative solver for the Inverse Kinematics problem». [En línea]. Disponible en: https://www.andreasaristidou.com/FABRIK

[4] Wikipedia, «Inverse Kinematics — Heuristic Methods». [En línea]. Disponible en: https://en.wikipedia.org/wiki/Inverse_kinematics#Heuristic_methods

[5] Nocturnailed Community, «NocKinematics — FABRIK library for Arduino/ESP32». [En línea]. Disponible en: https://github.com/Nocturnailed-Community/NocKinematics

[6] Blender Foundation, «Inverse Kinematics Constraint — Blender Manual». [En línea]. Disponible en: https://docs.blender.org/manual/en/latest/animation/constraints/tracking/ik_solver.html

[7] Autodesk, «Rotate Plane IK solver — Maya Help». [En línea]. Disponible en: https://help.autodesk.com/cloudhelp/2017/ENU/Maya/files/GUID-9942FFB5-65C2-46E2-B5A3-297667A9FB5D.htm

[8] Mozilla Developer Network, «3D collision detection». [En línea]. Disponible en: https://developer.mozilla.org/en-US/docs/Games/Techniques/3D_collision_detection